Machine Learning Methods for Finance

Unsupervised Learning - Dimension Reduction

Prof. Mike Aguilar
Economics Department and Fuqua School of Business, Duke University

https:/www.linkedin.com/in/mike-aguilar-econ

Outline

1. Dimension Reduction
   - PCA
   - Autoencoders

Motivation

- These notes accompany the Duke Economics course *Machine Learning for Finance*.
- The course focuses on how machine learning methods can be used to design, evaluate, and implement systematic investment strategies.
- Our goal is to introduce a range of ML techniques with enough mathematical structure to build intuition, while avoiding unnecessary formalism.
- Throughout the notes we use a simple return–momentum signal as a running example. Although momentum is not always the ideal illustration for every technique, using a single, familiar signal provides a consistent thread. This is especially true when we transition to real data and empirical implementation.
- The methods we discuss are broadly applicable across many signals, asset classes, and investment settings.

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Label assets as high/low momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
  Group assets by temporal behavior, then measure momentum
- **Supervised**
  Forecast returns based on persistence

Defining Momentum

There are many ways to define momentum. Here are two approaches that are not only common, but are designed in a way to support our pedagogical goals.

1. Jegadeesh and Titman: $mom_{i,t} = r_{i,t-12 \to t-2}$
2. Technical Analysis style: $mom_{i,t} = r_{i,t-2} - r_{i,t-12}$

Outline

1. Dimension Reduction
   - PCA
   - Autoencoders

Reducing the Dimension of Momentum

Q: How form a momentum trading signal using Dimension Reduction?

- Suppose we want to construct a momentum signal using a double moving-average crossover (or equivalently, short-horizon minus long-horizon returns).
- A key modeling choice is the definition of the *long* and *short* horizons.
- For example, the "long" horizon could plausibly be 11, 12, or 13 months, each yielding a slightly different signal.
- Rather than selecting a single horizon ex ante, we can compute momentum signals over *multiple* long horizons.
- Dimension Reduction provides a way to combine these correlated signals by reducing them to a single dominant "long-horizon" factor that captures the common variation.

Related work: Zakarmulin (2015), *Market Timing with Moving Averages*, applies similar ideas using moving averages of prices rather than returns.

Formalizing the Signal

A toy example that goes long(short) stocks with high(low) momentum

1. Compute
2. Repeat
3. Compute momentum as

$$
\begin{aligned}
s_{i,t} &= \mathbf{1}\left[p^{(+)}(X_{i,t}, Z_t)\right] - \mathbf{1}\left[p^{(-)}(X_{i,t}, Z_t)\right] \\
p^{(+)} &= 1[Mom_i \geq q_{.75}(Mom)] \\
p^{(-)} &= 1[Mom_i \leq q_{.25}(Mom)]
\end{aligned}
$$

- $X_{i,t}$: returns on asset i
- Z_t: returns for all other assets

Dimension Reduction Techniques

Motivation for the dimension reduction: Reduce the number of dimensions of the dataset while retaining the most important information.

A couple types of Dimension Reduction:

- Principal Component Analysis (PCA)
- Autoencoders

Note: There are several other dimension reduction techniques that fall under the rubric of supervised learning.

Outline

1. Dimension Reduction
   - PCA
   - Autoencoders

Reducing Dimension of Returns with PCA

1. Compute returns on N assets over p horizons (here $p = 13$):

$$R = \{R^{(13\mathrm{m})}, R^{(12\mathrm{m})}, \ldots, R^{(1\mathrm{m})}\}, \qquad R \in \mathbb{R}^{N \times p}.$$

2. Center each feature (column): for horizon j define

$$\bar{R}^{(j)} = \frac{1}{N} \sum_{i=1}^{N} R_i^{(j)}, \qquad \tilde{R}_{ij} = R_i^{(j)} - \bar{R}^{(j)}.$$

   Write the centered matrix $\tilde{R} \in \mathbb{R}^{N \times p}$.

3. Covariance of centered returns:

$$\Sigma = \frac{1}{N-1} \tilde{R}^{\top} \tilde{R} \in \mathbb{R}^{p \times p}.$$

4. Compute first principal direction (loading vector):

$$v_1 = \arg \max_{\|v\|=1} v^\top \Sigma v$$

where Lagrangian $\mathcal{L}(v, \lambda) = v^\top \Sigma v - \lambda(v^\top v - 1)$ gives the stationary condition

$$\Sigma v = \lambda v,$$

so v_1 is the eigenvector associated with the largest eigenvalue λ_1, and

$$\lambda_1 = v_1^\top \Sigma v_1$$

5. For asset i the first PC score (principal component) is

$$z_{i1} = \tilde{R}_{i\cdot}\, v_1 \qquad (\text{so } z_1 = \tilde{R} v_1 \in \mathbb{R}^N).$$

6. Rank-1 reconstruction (part explained by z_{i1}): $\hat{R}^{(1)} = z_1 v_1^\top \in \mathbb{R}^{N \times p}$
7. Residual (what z_{i1} did not explain): $U_1 = \tilde{R} - \hat{R}^{(1)}$
8. The second principal direction v_2 is the first principal direction of the residual matrix U_1 (equivalently the eigenvector of Σ with the second largest eigenvalue). Repeat to get $v_3, \ldots, v_p$.

Building a Signal with PCA

Q: How build a momentum signal with PCA?

1. Compute "Long Return Returns" as $\tilde{R}^{Long} = \{R^{(13m)}, R^{(12m)}, R^{(11m)}\}$ and
2. Repeat for
3. Compute momentum

$$
\begin{aligned}
s_{i,t} &= \mathbf{1}\Big[p^{(+)}(X_{i,t}, Z_t)\Big] - \mathbf{1}\Big[p^{(-)}(X_{i,t}, Z_t)\Big] \\
p^{(+)} &= 1[Mom_i \geq q_{.75}(Mom)] \\
p^{(-)} &= 1[Mom_i \leq q_{.25}(Mom)]
\end{aligned}
$$

- $X_{i,t}$: returns on asset i
- Z_t: returns for all other assets

Outline

1. Dimension Reduction
   - PCA
   - Autoencoders

How Autoencoders Work

Goal: Learn a low–dimensional representation z of the data x that captures its essential structure.

Architecture:

$$x \xrightarrow{\text{Encoder } f_\theta} z \xrightarrow{\text{Decoder } g_\phi} \hat{x}$$

Optimization Problem (Unsupervised):

$$\min_{\theta,\phi} \sum_{i=1}^{N} \|x_i - g_\phi(f_\theta(x_i))\|^2$$

- No labels: the neural network is trained to *reconstruct its own input*.
- Encoder $f_\theta(\cdot)$
- Decoder $g_\phi(\cdot)$
- The bottleneck dimension $k < p$
- If f and g are linear and the loss is MSE, the solution
- With nonlinear activations, autoencoders learn *nonlinear* low-dimensional structure.

Autoencoder Mappings

Encoders / Decoders differ in # of layers and activation functions

A single hidden layer encoder may take the form:

$$z = f_\theta(x) = \sigma(W_2\,\phi(W_1 x + b_1) + b_2)\,, \qquad f_\theta : \mathbb{R}^p \to \mathbb{R}^k,$$

where

- W_1, W_2
- b_1, b_2
- ϕ is the
- σ is

Common activation functions:

- **Linear:**
- **ReLU:**
- **tanh:** $\phi(x) = \tanh(x) = \dfrac{e^x - e^{-x}}{e^x + e^{-x}}$
- **ELU:** $\phi(x) = \begin{cases} x, & x > 0, \\ \alpha(e^x - 1), & x \le 0, \end{cases} \qquad (\alpha > 0)$

Building a Signal with Autoencoders

Q: How build a momentum signal with Autoencoder?

1. Compute "Long Return Returns" as $\tilde{R}^{Long} = \{\tilde{R}^{(13m)}, \tilde{R}^{(12m)}, \tilde{R}^{(11m)}\}$.
2. Repeat for
3. Compute momentum

$$
\begin{aligned}
s_{i,t} &= \mathbf{1}\Big[p^{(+)}(X_{i,t}, Z_t)\Big] - \mathbf{1}\Big[p^{(-)}(X_{i,t}, Z_t)\Big] \\
p^{(+)} &= 1[Mom_i \geq q_{.75}(Mom)] \\
p^{(-)} &= 1[Mom_i \leq q_{.25}(Mom)]
\end{aligned}
$$

- $X_{i,t}$: returns on asset i
- Z_t: returns for all other assets